Secure Computing

Assignment 2 — Part 2

DevSecOps, Threat Modelling,
Vulnerability Assessment & Supply Chain Security

Student:	Dylan Walt
Student No.:	2726341
Module:	Secure Computing
Assessment:	Assignment 2 — Part 2
Year Level:	Fourth Year, Semester 1
Date:	May 19, 2026

Submitted in partial fulfilment of the requirements for the module on Secure Computing.

Contents

Question 1: DevSecOps Critical Evaluation

Assessment Requirement

Critically evaluate the effectiveness of DevSecOps in reducing modern application security risks. Maximum: 1 200 words.

Introduction

DevSecOps represents a maturation of the DevOps model, embedding security as a shared responsibility across development, security, and operations teams throughout the entire software delivery lifecycle (SDL). Despite increasing adoption of CI/CD pipelines and automated security tooling, organisations continue to suffer significant security breaches. The 2021 Log4Shell vulnerability and the 2020 SolarWinds supply chain compromise illustrate that technical tooling alone is insufficient. This essay critically evaluates DevSecOps effectiveness across six dimensions: Shift Left Security, the limitations of automation, organisational culture, false positives and tool overload, open-source dependency risk, and the necessity of continuous monitoring.

Shift Left Security

"Shifting Left" denotes the practice of integrating security review, testing, and architectural analysis earlier in the SDL, reducing the cost and consequence of vulnerability discovery. Research by IBM Systems Sciences Institute (2012) demonstrated that a defect identified at the design phase costs up to 30 times less to remediate than one discovered in production. In practice, Shift Left manifests through embedding Static Application Security Testing (SAST) tools such as SonarLint directly into developer IDEs, conducting threat modelling exercises during sprint planning using frameworks such as STRIDE or PASTA, and enforcing security-focused pull request review policies. Microsoft's Security Development Lifecycle (SDL) is a canonical example: by mandating threat modelling, static analysis, and security sign-off at every development milestone, Microsoft demonstrably reduced the vulnerability density of its products (Howard and Lipner, 2006). However, Shift Left Security is effective only when developers receive adequate security training and when tooling generates actionable, low-noise guidance. Without security awareness education, Shift Left degrades into a series of automated gates that developers learn to suppress rather than remediate, undermining the very premise of the approach (Johnson et al., 2013).

Why Automation Alone is Insufficient

Automated security tooling accelerates vulnerability discovery at scale, yet it carries well-documented limitations that preclude it from replacing human judgement. SAST tools operate on predefined rule sets and are unable to detect logical business-logic vulnerabilities, authentication flaws rooted in contextual misuse, or race conditions that require semantic understanding of application behaviour (Livshits and Lam, 2005). The 2020 SolarWinds SUNBURST supply chain attack is emblematic of automation's blind spots: malicious code was surgically inserted into a trusted build pipeline at a stage where automated scanners classified the output as a legitimate, signed artefact (CISA, 2021). No static or dynamic scanner triggered an alert because the attack exploited the trustworthiness of the pipeline itself. Similarly, Dynamic Application Security Testing (DAST) tools cannot replicate the creative, context-driven reasoning of a skilled penetration tester exploring novel attack chains. The National Institute of Standards and Technology (2018) Cybersecurity Framework explicitly positions automated detection controls within a broader, human-governed risk management programme, emphasising that automation must augment security professionals rather than supplant them.

The Importance of Organisational Culture

Before it is a technical transformation, DevSecOps is a cultural one. Effective implementation requires dismantling the traditional model in which a siloed security function gates software releases, replacing it with a shared-ownership model in which every team member bears proportionate security responsibility (Kim et al., 2016). Forsgren et al. (2018) demonstrated through large-scale empirical research that high-performing software organisations exhibit generative cultures characterised by psychological safety, high cooperation, and blameless post-incident reviews — cultural properties that are prerequisite for sustainable security improvement. Organisations that deploy DevSecOps tooling without corresponding cultural alignment frequently encounter developer resistance, security team marginalisation, and chronic under-investment in security skills development. The 2019 Capital One breach, in which a misconfigured AWS Web Application Firewall enabled a server-side request forgery (SSRF) attack resulting in the exfiltration of over 100 million customer records, reflected not merely a technical failure but an organisational failure in cloud configuration governance, access management review, and security accountability structures (United States Senate Permanent Subcommittee on Investigations, 2019).

Challenges of False Positives and Tool Overload

A persistent operational challenge in DevSecOps adoption is the generation of excessive false positive alerts by automated security tools. Johnson et al. (2013) found that a primary reason developers dismiss static analysis results is the perceived inaccuracy of tool output, with false positive rates exceeding 50% in large enterprise codebases. When the signal-to-noise ratio of a security tool is poor, development teams rationally develop alert fatigue, increasing the probability that genuine vulnerabilities are dismissed alongside spurious warnings. Compounding this, organisations frequently adopt multiple overlapping security tools simultaneously — SAST, DAST, Software Composition Analysis (SCA), container image scanning, infrastructure-as-code scanning, and secret detection — without centralised orchestration. The resulting fragmented security landscape makes triaging, prioritising, and tracking vulnerabilities operationally unmanageable, leading to security debt accumulation rather than reduction. Platforms such as DefectDojo can consolidate findings, but organisations still need deliberate tool rationalisation, clear remediation SLAs, and executive accountability; otherwise low-confidence alerts obscure actionable risk.

The Impact of Vulnerable Open-Source Dependencies

Modern applications are substantially composed of open-source libraries, creating systemic software supply chain risk. The 2021 Log4Shell vulnerability (CVE-2021-44228) in the widely-deployed Apache Log4j logging library exposed an estimated three billion devices globally and required emergency cross-industry patching at unprecedented scale (Chen et al., 2022). The 2017 Equifax data breach similarly exploited CVE-2017-5638, a critical known vulnerability in the Apache Struts web framework that had remained unpatched for approximately two months due to inadequate dependency lifecycle management, ultimately exposing the personal and financial data of 147 million consumers (Federal Trade Commission, 2019). SCA tools such as OWASP Dependency-Check, Snyk, and GitHub Dependabot provide automated identification of known vulnerabilities by querying the National Vulnerability Database (NVD) and the GitHub Advisory Database. However, SCA is inherently reactive, detecting established CVEs while remaining blind to zero-day vulnerabilities and deliberately malicious packages, as demonstrated by the 2021 compromise of the `ua-parser-js` npm package. Organisations must therefore complement automated SCA with Software Bill of Materials (SBOM) generation per NTIA guidance (National Telecommunications and Information Administration, 2021), dependency version pinning, and supply chain security assessments.

Why Continuous Monitoring Remains Necessary After Deployment

Deploying an application does not conclude the security lifecycle. Post-production threats including runtime exploits, newly disclosed zero-day vulnerabilities, insider threats, and advanced persistent threats (APTs) necessitate continuous monitoring capabilities independent of the pre-deployment pipeline. The inadequacy of deployment-gate security is starkly demonstrated by the Equifax breach, in which attackers exfiltrated data for 78 days before detection (Federal Trade Commission, 2019), and by the 2013 Target breach, in which security monitoring tools generated alerts that were not acted upon by operations staff. Tools such as Falco for container runtime security, AWS GuardDuty for cloud-native threat detection, and SIEM platforms such as Splunk or Elastic Security provide real-time production visibility. National Institute of Standards and Technology (2011) mandates Information Security Continuous Monitoring (ISCM) as a fundamental governance requirement, emphasising that threat landscapes evolve independently of the software delivery cycle. Effective DevSecOps programmes treat continuous monitoring as an integral pipeline stage, with defined alerting thresholds, automated incident response runbooks, and feedback loops that propagate production findings upstream into backlog prioritisation and threat model updates. Quantitatively, Forsgren et al. (2018) found that elite software delivery organisations restore service from failures 2,604 times faster than low-performing counterparts — an outcome achievable only through comprehensive observability and rapid-response monitoring. This underscores that continuous monitoring is not merely a reactive security control but a strategic organisational capability that enables evidence-based improvement and informed risk governance.

Conclusion

DevSecOps can materially reduce application security risk by embedding security controls throughout the software delivery lifecycle, but only when supported by cultural change, informed human review, and continuous monitoring. Treating DevSecOps as a tooling procurement exercise leaves organisations exposed; sustained improvement requires accountable ownership, prioritised remediation, and feedback loops that keep pace with evolving threats.

Approximate word count: 1 165 words

Question 2: Scenario-Based Design: PayWave Digital DevSecOps Pipeline

Scenario Summary

PayWave Digital is a Johannesburg-based FinTech startup launching a cloud-native digital payments platform (microservices, Docker, AWS, GitHub Actions). Key risks identified during due-diligence: vulnerable OSS dependencies, secrets committed to GitHub, absent automated security testing, and insufficient runtime monitoring.

2.1 Security Risk Analysis

Before designing the DevSecOps pipeline, a structured analysis of PayWave's identified risks is essential:

1. **Vulnerable open-source dependencies.** Heavy reliance on open-source libraries for authentication, logging, payment processing, and API management creates a broad attack surface. Libraries may contain known CVEs and are typically not audited before inclusion (Snyk, 2023).
2. **Secrets management failures.** A developer accidentally committed live API credentials to a private GitHub repository. Without automated secret detection and centralised secrets management, credentials may persist in version history and propagate across environments.
3. **Absent automated security testing.** No SAST, DAST, or SCA tooling is integrated into the GitHub Actions pipeline. Vulnerabilities are not discovered until after deployment or, worse, after exploitation.
4. **Insufficient runtime monitoring.** Without real-time production monitoring, the team cannot detect anomalous API access patterns, container escapes, data exfiltration, or fraud-related anomalies indicative of active compromise.
5. **Container and infrastructure risks.** Docker containers without image scanning may run with known OS-level or library-level vulnerabilities. Infrastructure- as-code misconfigurations can expose internal services to the public internet.

Table 1 maps each identified risk to the specific pipeline phase and security control that addresses it, demonstrating how the proposed pipeline directly mitigates the concerns raised during due diligence.

Table 1: PayWave Due-Diligence Risks Mapped to Pipeline Controls

Risk Identified	Pipeline Phase	Security Control
Vulnerable OSS dependencies	Build	SCA: Snyk / OWASP Dependency-Check; build fails on CVSS ≥ 7
Secrets committed to GitHub	Code	Pre-commit hooks: GitLeaks / truffleHog detect secrets before push
No automated security testing	Code / Build / Test	SAST (Semgrep), SCA (Snyk), DAST (OWASP ZAP) integrated into pipeline
Insufficient runtime monitoring	Monitor	AWS GuardDuty, Falco, CloudWatch with automated PagerDuty alerting
Container & IaC misconfigurations	Build / Deploy	Container scanning (Trivy), IaC scanning (Checkov / tfsec)

2.2 DevSecOps Pipeline Diagram

Figure 1 presents the proposed secure DevSecOps pipeline for PayWave Digital, covering all seven phases of the software delivery lifecycle with embedded security controls.

Figure 1: PayWave Digital — Proposed Secure DevSecOps CI/CD Pipeline

2.3 Security Controls by Phase

Phase 1 — Plan

Every feature sprint begins with a structured threat modelling session using the STRIDE framework to enumerate threats applicable to PayWave's payment APIs and microservice interactions. Security requirements derived from threat models, PCI DSS obligations (PCI Security Standards Council, 2022), and the OWASP ASVS (OWASP Foundation, 2021a) are documented as acceptance criteria in Jira tickets, ensuring that security is a first-class functional requirement rather than an afterthought.

Phase 2 — Code

SAST is integrated into the GitHub Actions workflow and developer IDEs via SonarLint. Every commit triggers Semgrep rules customised for Python, Node.js, and Java microservices. Critically for PayWave, pre-commit hooks enforced via `.pre-commit-config.yaml` run GitLeaks and truffleHog to detect hard-coded secrets, API keys, and private credentials before they enter version control — directly addressing the incident identified during investor due diligence. Peer code review is mandated for all changes to security-sensitive modules (authentication, payment processing, API gateways).

Phase 3 — Build

Software Composition Analysis using Snyk and OWASP Dependency-Check scans all third-party libraries and transitive dependencies against the NVD and GitHub Advisory Database on every build. Container images are scanned with Trivy (Aqua Security, 2023) for OS-level and library-level vulnerabilities before being pushed to the container registry. An SBOM is generated using Syft for every release artefact, enabling PayWave to respond rapidly to future supply chain vulnerability disclosures. Builds with critical or high-severity unresolved findings fail automatically.

Phase 4 — Test

DAST is performed against the staging environment using OWASP ZAP in automated scan mode within the GitHub Actions pipeline, targeting all publicly exposed REST API endpoints and the customer-facing web application. API security testing using Postman/Newman or 42Crunch validates that JWT authentication, authorisation boundaries, and rate-limiting controls function correctly. Quarterly manual penetration testing by an external firm provides deeper coverage of business-logic vulnerabilities that automated

tools cannot reliably detect.

Phase 5 — Release

Automated security quality gates prevent promotion to production unless all critical and high-severity findings from SAST, SCA, and DAST phases are resolved or formally accepted. Release artefacts (container images, Helm charts) are cryptographically signed using Sigstore's `cosign`, ensuring supply chain integrity from build to deployment.

Phase 6 — Deploy

HashiCorp Vault (HashiCorp, 2023) provides centralised secrets management, injecting database credentials, payment gateway API keys, and internal service tokens into running containers at runtime via the Vault Agent Sidecar — eliminating hard-coded credentials entirely. Infrastructure-as-code templates (Terraform/CloudFormation) are scanned with Checkov and tfsec before deployment to detect misconfigurations (open security groups, public S3 buckets, missing encryption). AWS IAM roles follow least-privilege principles, enforced through automated policy validation.

Phase 7 — Monitor

AWS CloudWatch and GuardDuty provide cloud-native threat detection, with Falco installed as a Kubernetes DaemonSet for container runtime security monitoring. Anomalous API access patterns, authentication anomalies, and unusual data transfer volumes trigger automated alerts routed to the Security Operations team via PagerDuty. Findings feed back into the threat model and backlog through a weekly security review meeting, closing the continuous improvement loop illustrated in Figure 1.

Question 3: Threat Modelling: STRIDE Analysis for Healthcare API Platform

Scenario

An organisation is deploying a cloud-native healthcare API platform that stores and exposes patient medical records. The STRIDE framework is applied to identify one realistic threat per category, its CIA impact, and a recommended mitigation.

3.1 STRIDE Threat Analysis

STRIDE Category	Threat Scenario	CIA Impact	Mitigation Strategy
Spoofing Identity	An attacker obtains a stolen OAuth 2.0 access token via phishing and replays it to authenticate as a legitimate clinician, accessing the patient records API without authorisation.	**Confidentiality:** Unauthorised access to protected health information (PHI) violates POPIA (Republic of South Africa, 2013) and HIPAA, risking regulatory sanction and patient harm.	Implement short-lived JWT tokens (15-min expiry) with rotating refresh tokens; enforce multi-factor authentication (MFA) for all API access; deploy token revocation lists and anomalous usage detection.

STRIDE Category	Threat Scenario	CIA Impact	Mitigation Strategy
Tampering with Data	A man-in-the-middle (MITM) attacker intercepts unencrypted API traffic between a hospital system and the cloud API, modifying a patient's medication dosage field before it reaches the records database.	**Integrity:** Falsified medical records could lead to incorrect clinical decisions, medication errors, and patient harm.	Enforce TLS 1.3 with HSTS for all API communications; apply HTTP Strict Transport Security (HSTS); implement digital signatures (HMAC-SHA256) on critical record payloads to detect in-transit tampering.
Repudiation	A healthcare professional accesses and modifies a patient's diagnosis records, then later claims the action was unauthorised or performed by another user, exploiting an absence of reliable audit trails.	**Integrity:** Without non-repudiation, accountability is impossible; investigations into data misuse or clinical negligence cannot be substantiated.	Implement immutable, cryptographically signed audit logs stored in a tamper-evident ledger (e.g., AWS CloudTrail with log file validation); include actor identity, timestamp, and record hash in every write event.

STRIDE Category	Threat Scenario	CIA Impact	Mitigation Strategy
Information Disclosure	An authenticated low-privilege patient portal user exploits an Insecure Direct Object Reference (IDOR) vulnerability by incrementing record IDs in API requests, successfully retrieving other patients' complete medical histories.	**Confidentiality:** Mass exposure of third-party PHI constitutes a notifiable data breach under POPIA Section 22 (Republic of South Africa, 2013), incurring regulatory penalties and reputational damage.	Enforce object-level authorisation on every API endpoint; implement Attribute-Based Access Control (ABAC) ensuring records are accessible only to their owning patient or authorised clinicians; conduct IDOR-specific security testing.
Denial of Service	An adversary launches a volumetric DDoS attack, flooding the patient records API with millions of requests per second, rendering the platform unavailable to clinical staff during an emergency response scenario.	**Availability:** Inability of clinical staff to access patient records in time-critical situations directly endangers patient lives, constituting a severe patient safety incident.	Deploy AWS WAF with rate-based rules and AWS Shield Advanced for DDoS mitigation; implement API gateway-level rate limiting and circuit breakers; configure auto-scaling with surge protection; utilise a CDN to absorb volumetric traffic.

STRIDE Category	Threat Scenario	CIA Impact	Mitigation Strategy
Elevation of Privilege	An attacker exploits a broken function-level authorisation vulnerability, sending an API request with a manipulated `role` field to escalate from a read-only patient user to an administrator, gaining full CRUD access to all patient records.	**Confidentiality & Integrity:** The attacker can read, modify, and delete arbitrary patient records, causing data integrity violations and mass PHI exposure simultaneously.	Enforce Role-Based Access Control (RBAC) validated server-side on every request, independent of client-supplied parameters; apply the principle of least privilege; use established authorisation frameworks (OPA/Casbin); include privilege escalation scenarios in regular penetration testing.

3.2 Why Threat Modelling Must Precede Penetration Testing

Threat modelling and penetration testing serve complementary but fundamentally different purposes in a security programme. Several reasons justify performing threat modelling *before* penetration testing:

- **Proactive versus reactive posture.** Threat modelling is a proactive, design-time activity that systematically enumerates potential attack vectors before any code is written. Penetration testing is reactive and binary — it probes a completed system for exploitable flaws. By the time a penetration tester identifies an architectural flaw (such as missing authentication on an internal API endpoint), fixing it may require significant rework of live systems (Howard and Lipner, 2006).
- **Cost of remediation.** Architectural security flaws identified during design are orders of magnitude cheaper to remediate than those discovered post- deployment. A design-level threat model can redirect architectural decisions before a single line of code is committed (IBM Systems Sciences Institute, 2012).
- **Completeness and coverage.** Penetration testing is inherently time- bounded and samples the attack surface rather than comprehensively covering it. Threat modelling systematically enumerates all trust boundaries, data flows, and assets, producing a

structured register of threats against which countermeasures can be verified. Penetration testing then validates whether the countermeasures are correctly implemented.

- **Design influence.** Many critical security controls (separation of concerns, privilege separation, data classification, encryption boundaries) can only be meaningfully influenced at the architecture and design stage. A penetration test on a deployed system cannot change its fundamental trust model.
- **Informed scoping.** Threat models produce structured threat registers that allow penetration testers to focus effort on the highest-risk attack paths, improving both the efficiency and depth of testing engagements.

In summary, threat modelling defines *what* should be secure and *why*, while penetration testing validates *whether* the implemented controls achieve that goal. Neither substitutes for the other; but threat modelling must come first.

Question 4: Practical DevSecOps Lab: OWASP ZAP Vulnerability Assessment

4.1 Introduction

This report documents a Dynamic Application Security Testing (DAST) exercise performed using OWASP ZAP (OWASP Foundation, 2023b) against the OWASP Juice Shop (Kimminich, 2023) — a deliberately vulnerable web application designed for security training. The objective was to demonstrate automated vulnerability discovery within a DevSecOps context, identify real-world application security weaknesses, and evaluate the effectiveness of DAST as a pipeline control. The scan was performed on **11 May 2026** against a locally hosted Juice Shop instance at `http://localhost:3000`. The assessment identified one **High**-risk vulnerability (SQL Injection) and multiple **Medium**-risk findings relating to Content Security Policy (CSP), cross-domain configuration (CORS), and session identifiers exposed in URLs.

4.2 Environment Setup

Target Application: OWASP Juice Shop

OWASP Juice Shop was deployed locally using Docker:

Listing 1: Deploy OWASP Juice Shop via Docker

```
docker pull bkimminich/juice-shop
docker run -d -p 3000:3000 bkimminich/juice-shop
```

The application was accessible at `http://localhost:3000`. Juice Shop is a Node.js/Express single-page application intentionally containing OWASP Top 10 vulnerabilities, making it an ideal DAST training target.

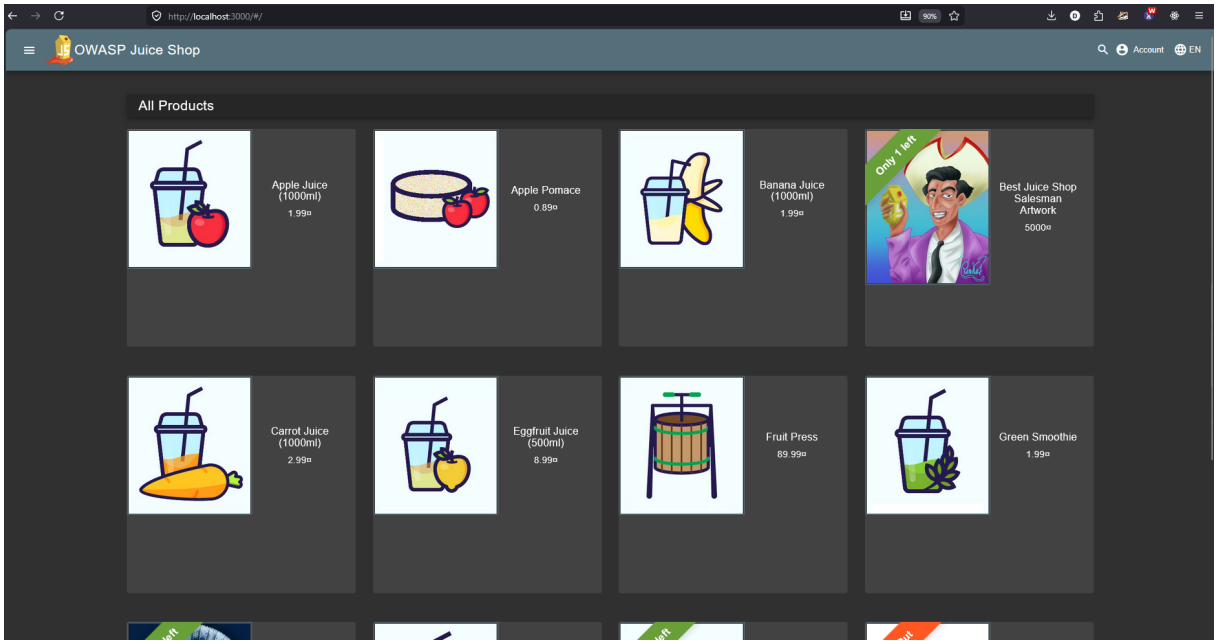

Figure 2: OWASP Juice Shop running locally at `http://localhost:3000`.

OWASP ZAP Installation

OWASP ZAP (version 2.14+) was downloaded from `https://www.zaproxy.org/download/` and installed on the host machine. The Community Edition provides all features required for this assessment.

Proxy Configuration

ZAP was configured as an intercepting proxy:

1. ZAP proxy listener set to `localhost:8080` (Tools → Options → Network → Local Servers/Proxies).
2. Firefox configured to use a manual proxy (`localhost:8080`) for HTTP and HTTPS traffic so that requests pass through ZAP for inspection and scanning.

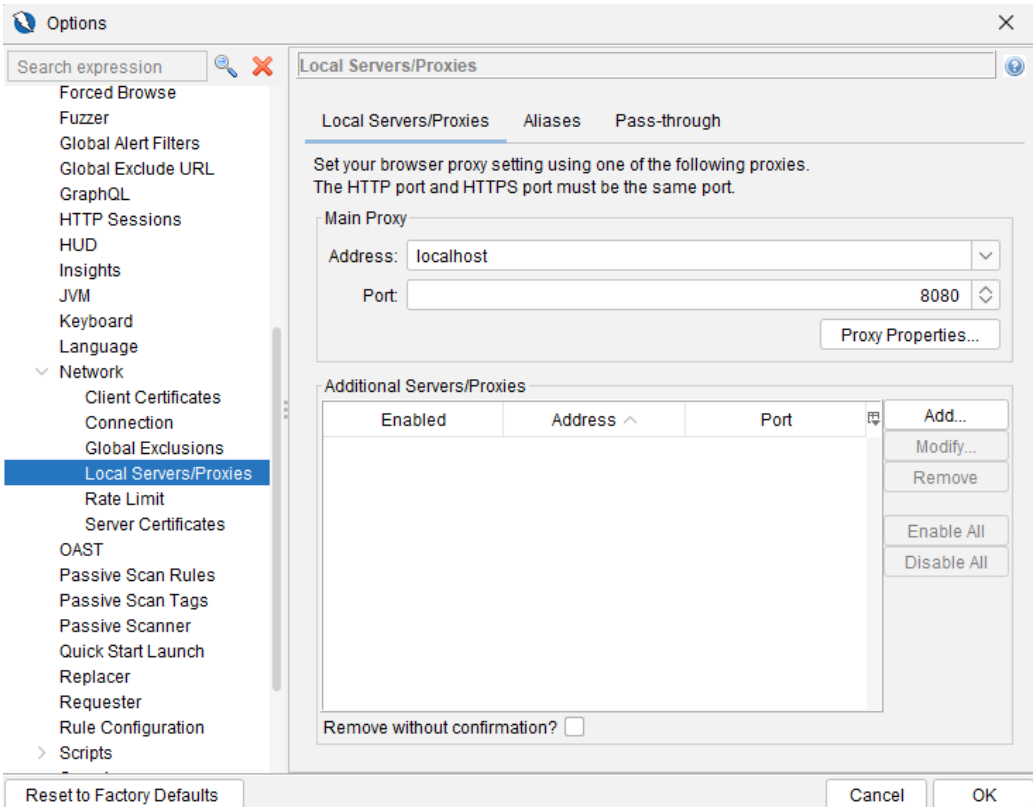

Figure 3: ZAP local proxy listener configured on `localhost:8080`.

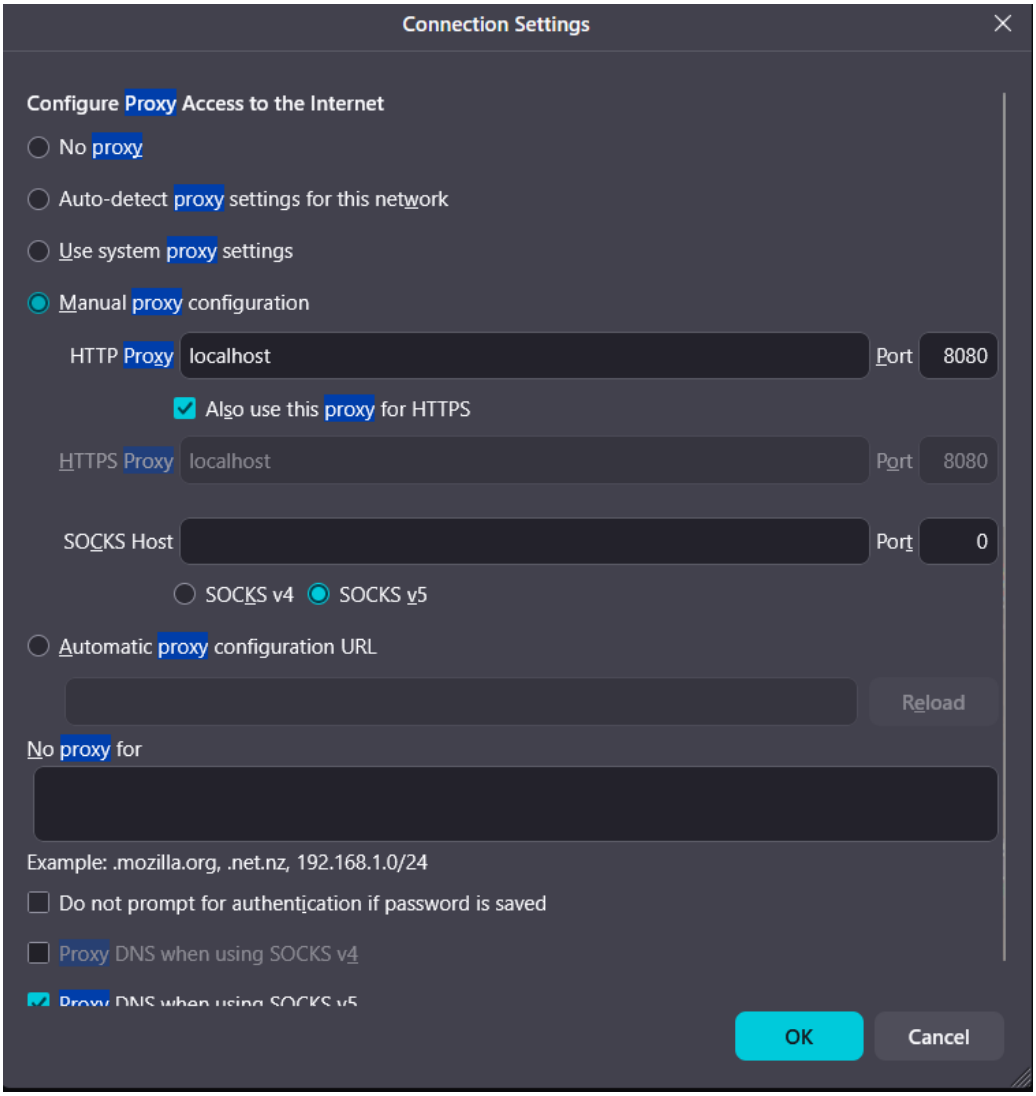

Figure 4: Firefox manual proxy settings pointing to `localhost:8080`.

4.3 Scan Configuration

Traditional Spider (Crawling)

The Traditional Spider was launched against `http://localhost:3000`:

- Navigate to: *Tools* → *Spider* → *New Scan*
- Starting URL: `http://localhost:3000`
- Results: **385 URLs found** and **274 nodes added** (including REST API endpoints, static assets, and application routes).

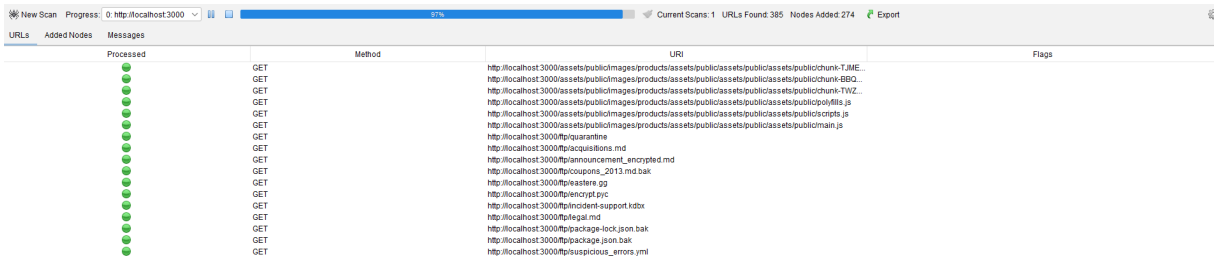

Figure 5: Traditional spider results showing discovered URLs and endpoints.

Note on AJAX Spider. Because Juice Shop is an Angular-based Single Page Application (SPA), OWASP ZAP's *AJAX Spider* is available as a complement to the Traditional Spider. The AJAX Spider executes JavaScript, follows client-side routes, and interacts with dynamic content, providing deeper coverage of modern SPA architectures. Combined use of both spiders is recommended for production-grade assessments against JavaScript-heavy applications.

Active Scan

Following the spider, an Active Scan was launched to probe discovered endpoints for vulnerabilities:

- Navigate to: *Tools* → *Active Scan* → *New Scan*
- Target: `http://localhost:3000`
- Scan Policy: Default Active Scan Policy
- The active scan generated large volumes of requests and flagged new alerts as it tested the discovered attack surface.

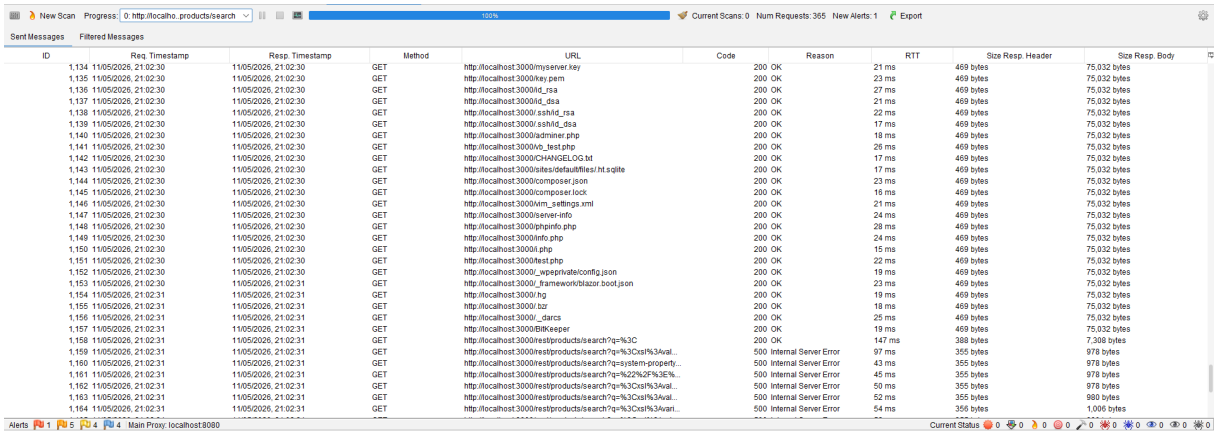

Figure 6: Active scanning traffic recorded in ZAP (requests and server responses).

4.4 Findings

The automated scan produced the following security findings. The complete Alerts list in ZAP is shown in Figure 7.

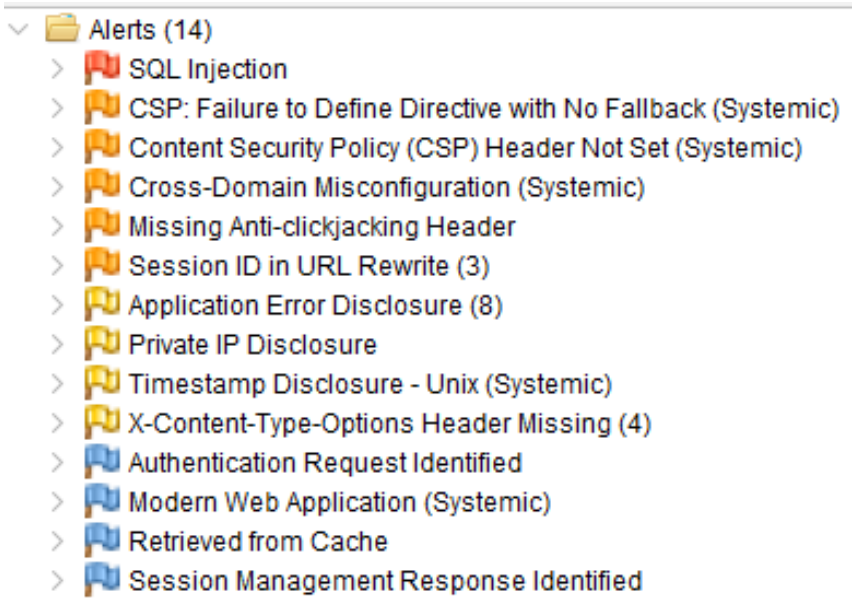

Figure 7: ZAP Alerts list for the Juice Shop scan (grouped by risk level).

Finding 1 — SQL Injection [HIGH RISK]

HIGH RISK: SQL Injection

Risk Level:	High
CWE:	CWE-89 — Improper Neutralisation of SQL Commands
OWASP 2021:	A03:2021 — Injection
Affected URL:	`GET http://localhost:3000/rest/products/search`
Parameter:	`q`
Evidence:	Crafted input triggered `HTTP/1.1 500 Internal Server Error` during scanning

Vulnerability Type. SQL Injection (SQLi) occurs when untrusted user-supplied input is incorporated directly into a database query without adequate sanitisation or parameterisation. An attacker can manipulate the query logic to bypass authentication, extract arbitrary data, modify records, or in some configurations, execute operating system commands (OWASP Foundation, 2021b).

Risk Impact. In this assessment, ZAP flagged a potential SQL injection in the Juice Shop product search endpoint `/rest/products/search` (parameter `q`). The alert evidence

shows server errors when crafted payloads were submitted, which is commonly associated with error-based SQLi detection and warrants manual verification. In a production payment platform such as PayWave, a confirmed SQL injection would enable:

- Unauthorised extraction of sensitive data (PII, accounts, transaction history).
- Integrity compromise by modifying records (e.g., balances, beneficiary details).
- Potential disruption of service availability through destructive queries.

The CVSS v3.1 base score for a remotely exploitable SQL injection is often **High–Critical** (worst-case **9.8**).

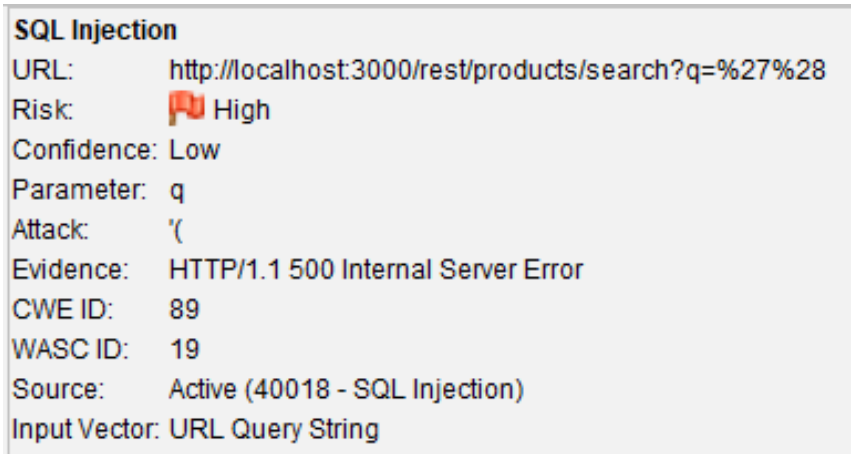

Figure 8: SQL Injection alert details raised by ZAP for `/rest/products/search`.

Mitigation Strategy.

- **Parameterised queries / prepared statements**: Replace all dynamic SQL string concatenation with parameterised queries using the ORM's bound-parameter API (e.g., Sequelize query builders with bound parameters rather than raw SQL strings).
- **Input validation**: Validate and sanitise all user inputs at the application boundary; reject inputs containing SQL metacharacters (`' - ;`).
- **Principle of least privilege**: The database user account used by the application should have minimal permissions (SELECT/INSERT/UPDATE only; no DROP or administrative functions).
- **Web Application Firewall (WAF)**: Deploy AWS WAF with SQL injection detection rules as a defence-in-depth layer.
- **SAST enforcement**: Enforce SAST rules (Semgrep `sql-injection` ruleset) in the CI pipeline to prevent injection-prone code patterns from merging.

Finding 2 — Content Security Policy (CSP) Header Not Set [MEDIUM RISK]

MEDIUM RISK: Content Security Policy (CSP) Header Not Set

Risk Level:	Medium
CWE:	CWE-693 — Protection Mechanism Failure
OWASP 2021:	A05:2021 — Security Misconfiguration
Affected URL:	`GET http://localhost:3000/socket.io/`
Evidence:	`Content-Security-Policy` response header absent

Vulnerability Type. The `Content-Security-Policy` (CSP) HTTP response header instructs browsers which sources of scripts, styles, frames, and other resources are trusted for a given page. When absent, browsers operate in their default permissive mode, permitting execution of injected scripts from any origin (OWASP Foundation, 2021b).

Risk Impact. Without a CSP header:

- Reflected and Stored XSS attacks can execute arbitrary JavaScript in victims' browsers, enabling session token theft, credential harvesting, and DOM manipulation.
- Malicious third-party scripts injected via supply chain compromise (e.g., a compromised npm CDN package) will execute unchallenged.
- Clickjacking attacks via `<iframe>` embedding remain possible.
- In a payment context such as PayWave, XSS combined with an absent CSP could enable attackers to silently redirect payment flows to attacker-controlled endpoints.

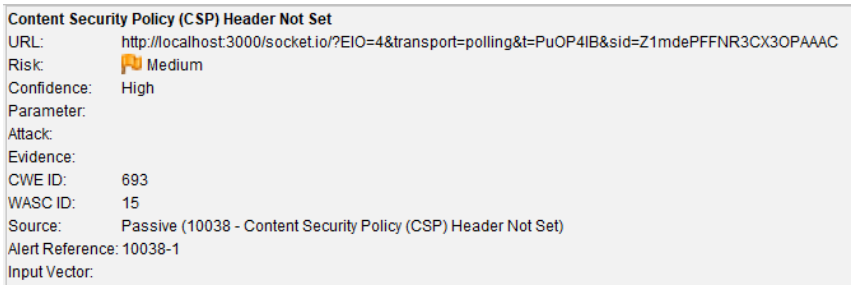

Figure 9: ZAP alert showing `Content-Security-Policy` header not set (Medium).

Mitigation Strategy.

- Add a `Content-Security-Policy` header to all HTTP responses, beginning with a restrictive policy such as:

```
Content-Security-Policy: default-src 'self';
  script-src 'self' https://trusted-cdn.com;
```

```
3   object-src 'none'; frame-ancestors 'none';
```

- Use `Content-Security-Policy-Report-Only` initially to audit policy effectiveness before enforcement.
- Combine with `X-Frame-Options: DENY` and `X-Content-Type-Options: nosniff` for defence-in-depth.
- Enforce CSP header presence in automated pipeline testing (ZAP passive scan rules).

Finding 3 — CSP: Failure to Define Directive with No Fallback [MEDIUM RISK]

MEDIUM RISK: CSP Directive Missing Fallback

Risk Level:	Medium
CWE:	CWE-693 — Protection Mechanism Failure
OWASP 2021:	A05:2021 — Security Misconfiguration
Affected URL:	`http://localhost:3000/assets/public/images/products`
Evidence:	CSP directive without fallback not defined (systemic)

Vulnerability Type. Some CSP directives (for example `frame-ancestors`) do not fall back to `default-src`. If such directives are omitted, the browser may apply permissive defaults, weakening protections against framing and related attacks.

Risk Impact.

- Increased exposure to clickjacking (framing) if `frame-ancestors` is not explicitly restricted.
- Reduced defence-in-depth against script injection where CSP should constrain trusted sources.
- In payment contexts, UI redressing and script execution can be leveraged to trick users into unauthorised actions.

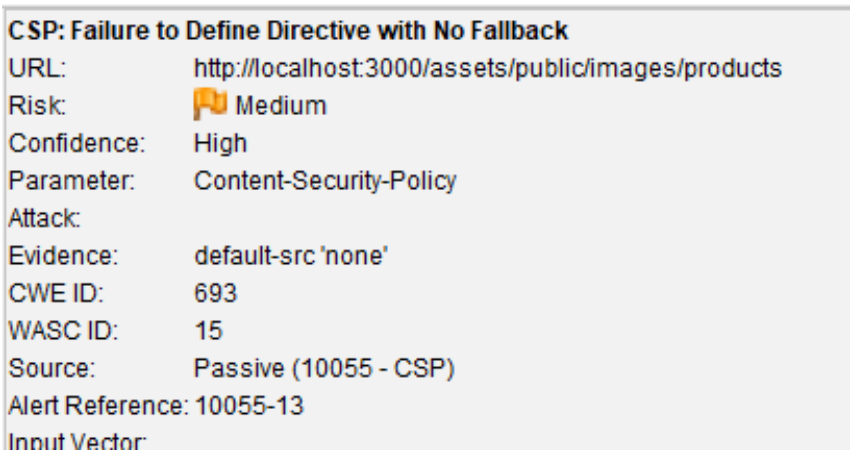

Figure 10: ZAP alert: CSP directive failure to define directive with no fallback (Medium).

Mitigation Strategy.

- Apply a complete CSP policy across the application, ensuring directives without fallback are explicitly set (e.g., `frame-ancestors 'none'`).
- Start with `Content-Security-Policy-Report-Only` to validate compatibility, then enforce once stable.
- Validate CSP consistency with automated DAST (ZAP passive scan) as a CI gate.

Finding 4 — Cross-Domain Misconfiguration (Permissive CORS) [MEDIUM RISK]

MEDIUM RISK: Cross-Domain Misconfiguration (CORS)

Risk Level:	Medium
CWE:	CWE-264 — Permissions, Privileges, and Access Controls
OWASP 2021:	A01:2021 — Broken Access Control
Affected URL:	`http://localhost:3000/chunk-*.js`
Evidence:	`Access-Control-Allow-Origin: *`

Vulnerability Type. Cross-Origin Resource Sharing (CORS) controls which origins (domains) are allowed to read responses in a browser context. A wildcard policy can be unsafe if applied to endpoints serving sensitive data or allowing authenticated requests.

Risk Impact.

- When enabled on sensitive API endpoints, permissive CORS can enable cross-site data access from attacker-controlled web origins.

- In a payments platform, this may contribute to exposure of customer or transaction data to malicious websites.

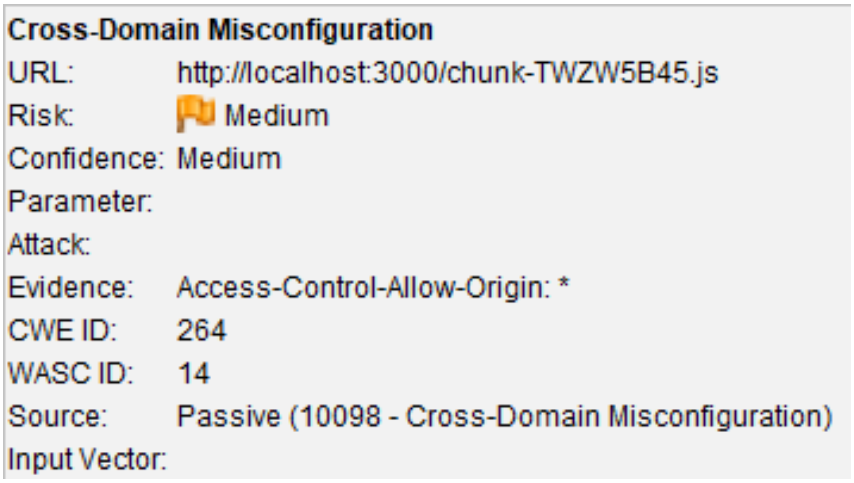

Figure 11: ZAP alert highlighting `Access-Control-Allow-Origin: *` (Medium).

Mitigation Strategy.

- Restrict allowed origins to an explicit allow-list and only enable CORS where it is required.
- Avoid enabling wildcard CORS on authenticated endpoints; rely on the Same Origin Policy by default.
- Re-test with ZAP after configuration changes to confirm remediation.

Additional Medium Finding — Session ID in URL Rewrite

ZAP also flagged **Session ID in URL Rewrite** (Medium), indicating that a session identifier was observed in the URL as a `sid` parameter. Session identifiers in URLs can leak via browser history, logs, and the `Referer` header, increasing the risk of session hijacking.

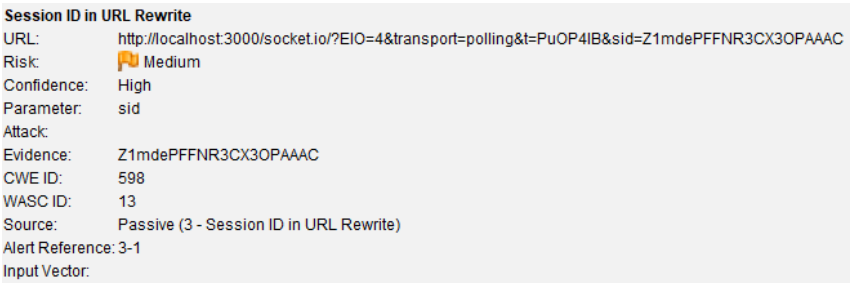

Figure 12: Session identifier observed in the URL (`sid` parameter).

Other Findings (Low/Informational)

ZAP also reported lower-priority findings that were not part of the required High/Medium minimum counts:

- **Timestamp Disclosure (Unix) [Low]:** a Unix timestamp value was disclosed in responses, which may help attackers infer build/deploy timing or internal behaviour.
- **Modern Web Application [Informational]:** identifies modern front-end behaviour (e.g., SPA scripting) and is primarily informational.

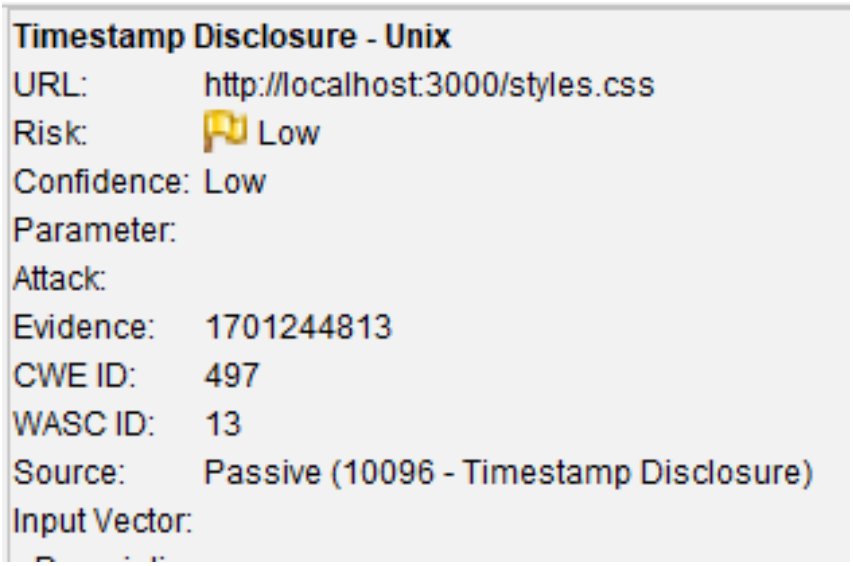

Figure 13: Low-risk finding: Timestamp Disclosure (Unix).

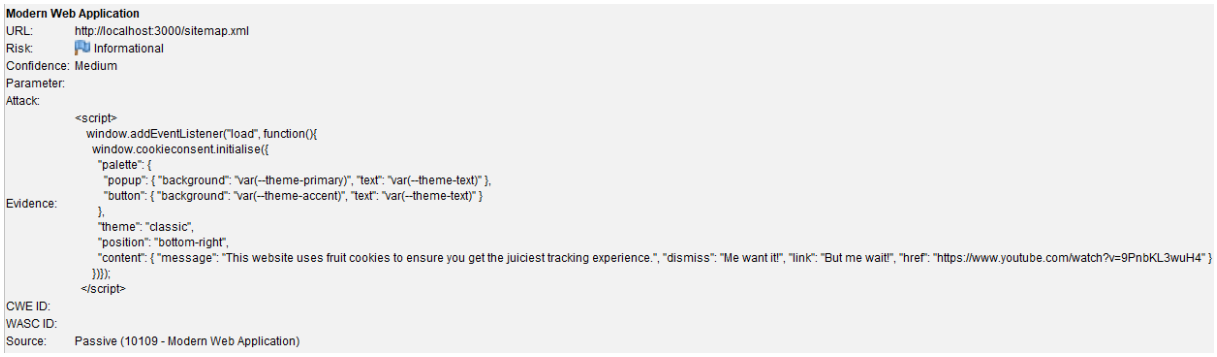

Figure 14: Informational finding: Modern Web Application indicator.

4.5 Risk Analysis Summary

#	Vulnerability	Risk	CVSS	Primary Impact
1	SQL Injection	**High**	9.8	Database compromise; unauthorised data access
2	CSP Header Not Set	**Medium**	6.1	Increased XSS/clickjacking exposure due to missing browser policy
3	CSP Directive Missing Fallback	**Medium**	5.3	Weak framing controls and reduced defence-in-depth
4	Cross-Domain Misconfiguration (CORS)	**Medium**	6.5	Cross-origin data exposure risk if applied to sensitive endpoints
5	Session ID in URL Rewrite	**Medium**	6.5	Session leakage via URLs (history/logs/`Referer`)

4.6 Mitigation Recommendations Summary

1. **Immediate (Critical):** Replace all raw SQL string interpolation with parameterised ORM queries. Apply input validation on all user-facing endpoints. Deploy WAF rules to detect and block SQL injection patterns. Re-test with ZAP after remediation.
2. **Short-term (High Priority):** Configure a complete `Content-Security-Policy` (including directives without fallback such as `frame-ancestors`) and ensure it is consistently applied across all application paths (including `/socket.io`). Validate header presence with ZAP passive scan rules.
3. **Short-term:** Restrict CORS to an explicit allow-list (avoid wildcard `Access-Control-Allow-Origin *` where not required) and prevent session IDs from appearing in URLs by using cookies or bearer tokens for session tracking.
4. **Ongoing:** Integrate ZAP Baseline Scan into the GitHub Actions pipeline to catch regressions on every PR. Schedule full Active Scans weekly against staging.

4.7 Reflection on DAST Effectiveness

This exercise demonstrates both the power and the limitations of DAST as a DevSecOps control. DAST is highly effective at identifying categories of vulnerability that manifest

at runtime — SQL injection, missing security headers, authentication weaknesses, and insecure cookie configurations. It requires no access to source code, making it applicable to third-party and legacy systems, and it tests the application in its actual deployed state rather than a static representation of the code.

However, DAST has meaningful limitations. Automated tools struggle with complex authentication flows, multi-step business logic attacks, and second-order injection vulnerabilities. The active scan may not achieve full coverage of a Single Page Application (SPA) due to the AJAX-heavy nature of many modern front-ends, and finding generation can include false positives that require manual verification. DAST is most effective when combined with SAST (which identifies source-level vulnerabilities earlier), SCA (which catches dependency risks), and manual penetration testing (which covers business logic and advanced attack chains) (Shahin et al., 2017). Within a mature DevSecOps pipeline, ZAP in Baseline mode provides a rapid, lightweight gate appropriate for every PR, while a full Active Scan is appropriate for pre-release staging validation.

4.8 Conclusion

In conclusion, OWASP ZAP proved effective at rapidly identifying impactful runtime vulnerabilities (including a High-risk SQL injection) and security misconfigurations (CSP and CORS) in a realistic web application. To maximise risk reduction, ZAP should be integrated into CI/CD as an automated DAST control and paired with complementary controls such as SAST, SCA, and ongoing monitoring to provide defence in depth.

Question 5: Software Supply Chain Security and GitHub Dependabot

5.1 Deliverable: GitHub Repository Link

Public GitHub repository link:

`https://github.com/dylanwalt/Secure_Computing_Ass_P_2`

5.2 What Are Vulnerable Dependencies?

Modern software applications are substantially composed of third-party and open-source libraries imported as dependencies. A **vulnerable dependency** is a library, framework, or package that contains a known security flaw (a CVE) that could be exploited by an attacker (OWASP Foundation, 2023a). Vulnerabilities may exist in a *direct* dependency explicitly declared in a manifest file (e.g., `package.json`, `pom.xml`, `requirements.txt`), or in a *transitive* (indirect) dependency — a library imported by a library. Research by Snyk (2023) found that the majority of vulnerabilities in production software originate from transitive dependencies that development teams may be entirely unaware of. The severity of a vulnerable dependency ranges from information disclosure (low) to remote code execution (critical), as exemplified by the Log4Shell (CVE-2021-44228) and Apache Struts (CVE-2017-5638) incidents discussed in Question 1.

5.3 Why Dependency Management Matters

Dependency management is a security-critical discipline for several reasons:

- **Attack surface expansion.** A typical Node.js application may have 500+ transitive dependencies. Each represents a potential attack vector that exists outside the development team's direct control (Snyk, 2023).
- **Supply chain attack risk.** Beyond known CVEs, adversaries increasingly target open-source packages with malicious code insertion (e.g., the 2021 `ua-parser-js` compromise), making proactive dependency monitoring essential.
- **Regulatory compliance.** PCI DSS Requirement 6.3.3 mandates that all software components be protected from known vulnerabilities. POPIA obligations around data security implicitly require organisations to remediate known exploitable flaws.

- **Rapid patch availability.** When a CVE is published, exploits often follow within hours ("exploitation in the wild"). Organisations with active dependency management can apply patches before exploitation; those without are exposed for days or weeks.
- **Cost of remediation.** Automated dependency updates applied proactively are substantially cheaper than emergency incident response following exploitation of an unpatched dependency.

5.4 GitHub Dependabot: Features and Enabling Alerts

What is Dependabot?

GitHub Dependabot (GitHub, 2024) is an automated Software Composition Analysis (SCA) tool built into GitHub that continuously monitors a repository's dependency manifest files and lock files, cross-referencing declared packages against the GitHub Advisory Database (GHAD) and NVD. When a known vulnerability is identified, Dependabot generates an automated **security alert** visible in the repository's Security tab, and optionally raises an automated **pull request** proposing an update to a patched version.

Dependabot supports a wide range of ecosystems including npm, pip, Maven, Gradle, Bundler, Composer, NuGet, Go modules, and GitHub Actions workflows, making it broadly applicable across modern polyglot software projects.

Enabling Dependabot Alerts (Step-by-Step)

The following steps enable (or verify) Dependabot alerts for a public GitHub repository:

1. Navigate to the target repository on `https://github.com`.
2. Click the **Settings** tab in the repository navigation bar (Figure 15).

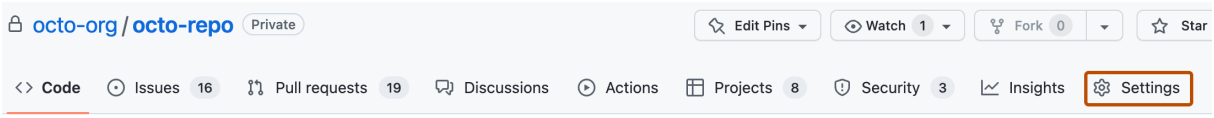

Figure 15: Repository navigation bar with the **Settings** tab highlighted (GitHub, 2026a).

3. In the **Security** section of the sidebar, click **Advanced Security**.
4. Under **Dependabot alerts**, click **Enable** (or confirm the status is already enabled). Figure 16 shows the **Disable** button, indicating that Dependabot alerts are currently enabled.

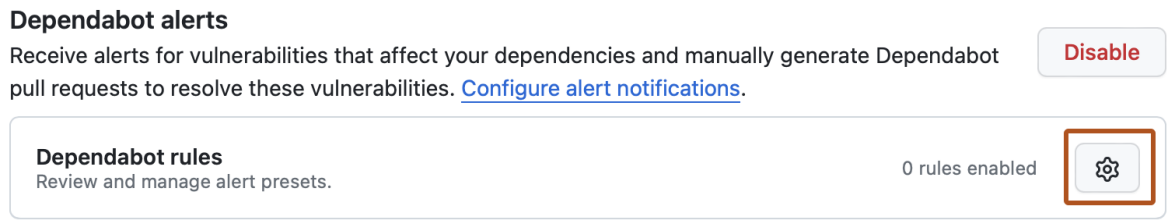

Figure 16: Dependabot alerts enabled in **Advanced Security** settings (GitHub, 2026b).

5. (Recommended) Enable **Dependabot security updates** so that GitHub can automatically propose remediation via pull requests when alerts are raised. When available, the alert details page provides a **Create Dependabot security update** action (Figure 17).

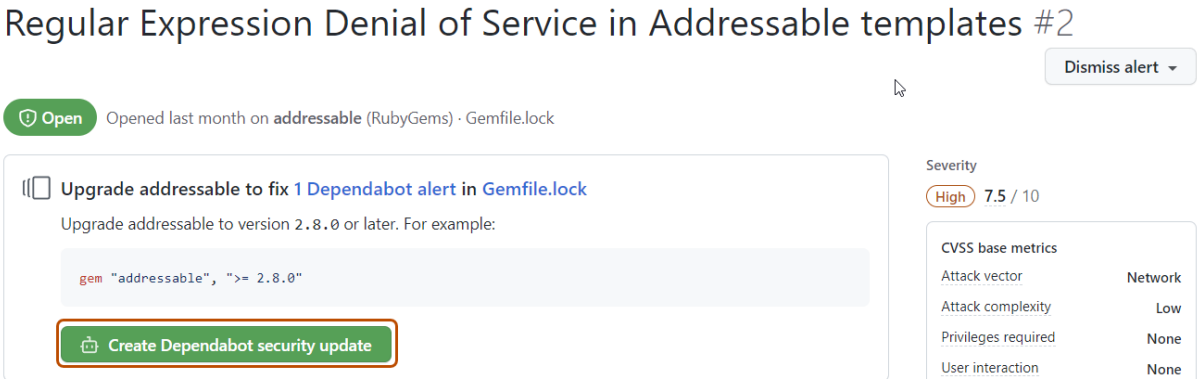

Figure 17: Dependabot alert details with the **Create Dependabot security update** button (GitHub, 2026c).

6. View alerts by navigating to the repository's **Security and quality** tab, then selecting **Dependabot** → **Vulnerabilities**. Figure 18 shows a typical list of alerts with severity labels.

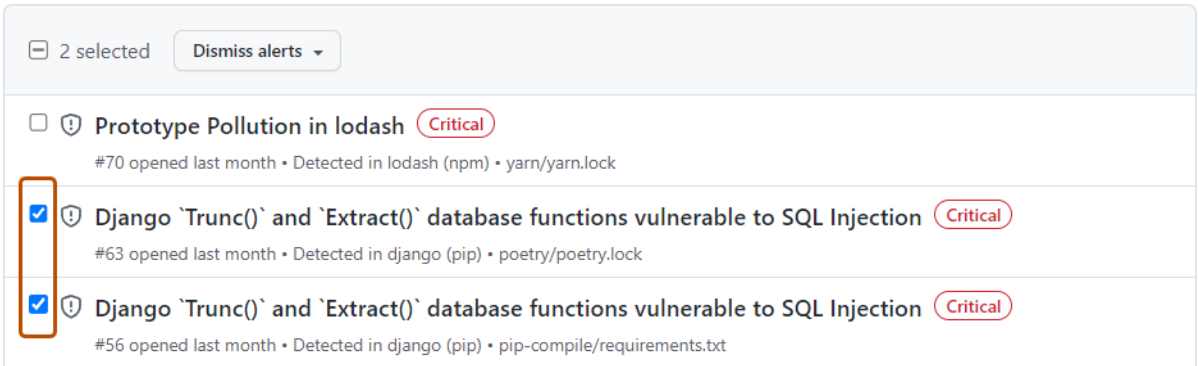

Figure 18: Example Dependabot alerts list with severity labels (GitHub, 2026c).

Interpreting Dependabot Alerts

Each Dependabot alert presents:

- **Package name and affected version(s)** — identifies the vulnerable component and the version range with the flaw.
- **CVE identifier and CVSS severity score** — contextualises the risk level (Low / Medium / High / Critical).
- **Patched version** — the version that resolves the vulnerability.
- **Dependency path** — whether the vulnerability is in a direct or transitive dependency, helping prioritise remediation effort.
- **Automated pull request** (if Dependabot security updates are enabled) — a ready-made branch updating the manifest to the patched version.

This information allows development teams to triage, prioritise (critical CVEs first), and remediate vulnerable dependencies with minimal manual research effort.

5.5 How SCA Tools Contribute to Secure CI/CD Pipelines

Software Composition Analysis tools integrate with CI/CD pipelines to provide automated, continuous dependency security scanning throughout the development lifecycle (OWASP Foundation, 2023a). Their contributions include:

- **Shift Left enforcement.** By scanning dependencies at build time (e.g., in a GitHub Actions `workflow.yml` step), SCA tools surface vulnerabilities before code merges to the main branch, preventing vulnerable components from reaching production.
- **Pipeline security gates.** SCA tools can be configured to *fail the build* when dependencies with CVSS scores above a threshold (e.g., $\geq$ 7.0 High) are detected, enforcing a non-negotiable remediation requirement.
- **SBOM generation.** Tools such as Syft and OWASP Dependency-Track generate Software Bills of Materials, providing a complete inventory of all components in a release artefact (National Telecommunications and Information Administration, 2021). SBOMs enable rapid impact analysis when new CVEs are disclosed.
- **Continuous monitoring.** Beyond build-time scanning, tools like Dependabot and Snyk continuously monitor dependency graphs against live vulnerability databases, generating alerts when new CVEs affecting already-deployed versions are published — even between releases.

- **Licence compliance.** SCA tools also identify open-source licences, helping organisations avoid licence incompatibilities (e.g., GPL in a commercial product).

A representative GitHub Actions workflow step using OWASP Dependency-Check:

Listing 2: GitHub Actions SCA step using OWASP Dependency-Check

```
name: Security Scan
on: [push, pull_request]

jobs:
  dependency-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run OWASP Dependency-Check
        uses: dependency-check/Dependency-Check_Action@main
        with:
          project: 'paywave-digital'
          path: '.'
          format: 'HTML'
          args: >
            --failOnCVSS 7
            --enableRetired

      - name: Upload Dependency-Check Report
        uses: actions/upload-artifact@v4
        with:
          name: dependency-check-report
          path: ${{ github.workspace }}/reports
```

This pipeline step automatically fails any build introducing a dependency with a CVSS score of 7 or higher, enforcing supply chain security as a mandatory quality gate consistent with DevSecOps principles.

Dependabot's automated update behaviour is further controlled through a `.github/dependabot.yml` configuration file committed to the repository. A representative configuration for a Node.js project with GitHub Actions workflow monitoring is shown below:

Listing 3: .github/dependabot.yml — Scheduled Dependabot update configuration

```
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
```

```
    labels:
      - "security"
      - "dependencies"
    open-pull-requests-limit: 10

  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

This configuration instructs Dependabot to check both `npm` packages and GitHub Actions workflow versions on a weekly schedule, automatically opening pull requests for available updates. Integrating this file into the repository ensures dependency monitoring is always active without requiring manual configuration through the GitHub UI.

References

Aqua Security (2023). Trivy: Container vulnerability scanner. GitHub. Available at: `https://github.com/aquasecurity/trivy`.

Chen, Y., Tian, Y., and Guo, Z. (2022). Understanding the Log4Shell vulnerability: Systemic impact and remediation. *IEEE Security & Privacy*, 20(3):28–37.

CISA (2021). SolarWinds and Active Directory/M365 compromise: Risk decisions for leaders. CISA Advisory AA21-008A. Cybersecurity and Infrastructure Security Agency.

Federal Trade Commission (2019). Equifax data breach settlement. FTC Publication. Available at: `https://www.ftc.gov/equifax-data-breach`.

Forsgren, N., Humble, J., and Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, Portland, OR.

GitHub (2024). About Dependabot security updates. GitHub Docs. Available at: `https://docs.github.com/en/code-security/dependabot`.

GitHub (2026a). Configuring Dependabot alerts. GitHub Docs. Available at: `https://docs.github.com/en/code-security/how-tos/secure-your-supply-chain/secure-you`

GitHub (2026b). Using GitHub preset rules to prioritize Dependabot alerts. GitHub Docs. Available at: `https://docs.github.com/en/code-security/how-tos/secure-your-supply-chain/manage-you`

GitHub (2026c). Viewing and updating Dependabot alerts. GitHub Docs. Available at: `https://docs.github.com/en/code-security/how-tos/manage-security-alerts/manage-deper`

HashiCorp (2023). Vault by HashiCorp: Secrets management and data encryption. Online. Available at: `https://www.vaultproject.io/`.

Howard, M. and Lipner, S. (2006). *The Security Development Lifecycle*. Microsoft Press, Redmond, WA.

IBM Systems Sciences Institute (2012). Relative cost of fixing defects. Technical report, IBM. Cost-benefit analysis of defect detection across lifecycle phases.

Johnson, B., Song, Y., Murphy-Hill, E., and Bowdidge, R. (2013). Why don't software developers use static analysis tools to find bugs? In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 672–681. IEEE Press.

Kim, G., Humble, J., Debois, P., and Willis, J. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press, Portland, OR.

Kimminich, B. (2023). OWASP juice shop. GitHub. Available at: `https://github.com/juice-shop/juice-shop`.

Livshits, V. B. and Lam, M. S. (2005). Finding security vulnerabilities in Java applications with static analysis. In *Proceedings of the 14th USENIX Security Symposium*, pages 271–286. USENIX Association.

National Institute of Standards and Technology (2011). Information security continuous monitoring (ISCM) for federal information systems and organizations. Technical Report SP 800-137, NIST, Gaithersburg, MD.

National Institute of Standards and Technology (2018). Framework for improving critical infrastructure cybersecurity, version 1.1. Technical Report NIST CSF v1.1, NIST, Gaithersburg, MD.

National Telecommunications and Information Administration (2021). The minimum elements for a software bill of materials (SBOM). NTIA Report. U.S. Department of Commerce.

OWASP Foundation (2021a). OWASP application security verification standard (ASVS) 4.0.3. Online. Available at: `https://owasp.org/www-project-application-security-verification-standard/`.

OWASP Foundation (2021b). OWASP top ten 2021. Online. Available at: `https://owasp.org/www-project-top-ten/`.

OWASP Foundation (2023a). OWASP dependency-check. Online. Available at: `https://owasp.org/www-project-dependency-check/`.

OWASP Foundation (2023b). OWASP ZAP — zed attack proxy. Online. Available at: `https://www.zaproxy.org/`.

PCI Security Standards Council (2022). Payment card industry data security standard (PCI DSS) v4.0. PCI SSC. Available at: `https://www.pcisecuritystandards.org/`.

Republic of South Africa (2013). Protection of personal information act 4 of 2013 (POPIA). Government Gazette No. 37067. Act No. 4 of 2013, Republic of South Africa.

Shahin, M., Babar, M. A., and Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. *IEEE Access*, 5:3909–3943.

Snyk (2023). State of open source security 2023. Snyk Annual Report. Available at: `https://snyk.io/reports/open-source-security/`.

United States Senate Permanent Subcommittee on Investigations (2019). Examining failures in the Capital One data breach. Senate Report. Homeland Security and Governmental Affairs Committee.